

Task 19

Crypto

No Salt, No Shame

Download Task Files

BEGINNER

15 Points

18


Crypto

To "secure" the maintenance logs, Virelia's gateway vendor encrypted every critical entry with AES-CBC—using the plant's code name as the passphrase and a fixed, all-zero IV. Of course, without any salt or integrity checks, it's only obscurity, not true security. Somewhere in those encrypted records lies the actual shutdown command.

Passphrase: **VIRELIA-WATER-FAC**

Download the encrypted log file attached to this task and get the flag!

Answer the questions below

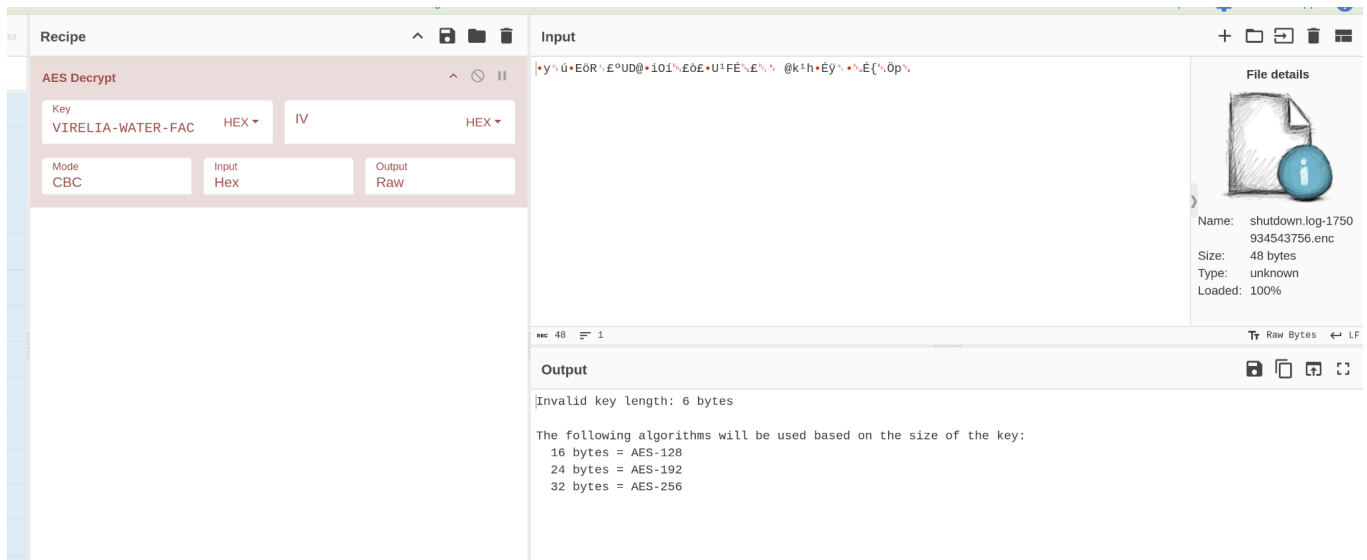
What is the flag?

Answer format: ***{*** _ * 16}

Submit

- Cipher: AES in CBC mode
- Key (Passphrase): VIRELIA-WATER-FAC
- IV: All-zero (`\x00 * 16`)
- No salt or integrity checks
- Encrypted file: `shutdown.log-1750934543756.enc` (48 bytes)
- File is exactly 48 bytes (3 AES blocks), consistent with AES block size (16 bytes)
- Prompt indicates passphrase used directly, with no mention of hashing, encoding, or key derivation functions

1. CyberChef:



- Attempted decryption with full passphrase (VIRELIA-WATER-FAC) failed due to invalid key length (17 bytes).
- Truncated to 16 bytes (VIRELIA-WATER-FA) — decryption returned only garbage characters.
- Padding variants (to 24 or 32 bytes) also failed — output was similarly invalid.

2. Python with PyCryptodome:

- Used `AES.new(key, AES.MODE_CBC, iv)` with:
 - Truncated key (16 bytes)
 - MD5 hash of full passphrase
 - Padded key versions (to 24 and 32 bytes)
 - Hex-encoded versions of passphrase

AES-CBC Decryption with Truncated Key (16 Bytes)

```
from Crypto.Cipher import AES

# Load ciphertext from file
with open("shutdown.log.enc", "rb") as f:
    ciphertext = f.read()

# AES setup
key = b"VIRELIA-WATER-FA" # 16 bytes
iv = b"\x00" * 16 # All-zero IV

# AES decryption
cipher = AES.new(key, AES.MODE_CBC, iv)
plaintext = cipher.decrypt(ciphertext)
```

```
# Attempt to decode (remove common padding bytes)
decoded =
plaintext.rstrip(b"\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10")

print(decoded.decode("utf-8", errors="ignore"))
```

Each variant resulted in either:

- Invalid key length errors, or
- Successful decryption of the ciphertext into non-ASCII, unstructured binary data (in all cases)

Try #2 openssl style

```
import hashlib
from Crypto.Cipher import AES

# Derive key using MD5 hash of passphrase
passphrase = b"VIRELIA-WATER-FAC"
key = hashlib.md5(passphrase).digest()
iv = b"\x00" * 16

with open("shutdown.log.enc", "rb") as f:
    ciphertext = f.read()

cipher = AES.new(key, AES.MODE_CBC, iv)
plaintext = cipher.decrypt(ciphertext)

decoded =
plaintext.rstrip(b"\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10")

print(decoded.decode("utf-8", errors="ignore"))
```

3. Hex Dump Analysis:

- Decrypted output was hex-dumped and analyzed for any readable patterns or ASCII sequences.

Hex Dump of Ciphertext or Decrypted Output

```
import binascii
```

```
# Use either raw ciphertext or decrypted output
data = plaintext # or use ciphertext if inspecting raw

hex_dump = binascii.hexlify(data).decode()

# Group output in 16-byte blocks for readability
for i in range(0, len(hex_dump), 32):
    print(hex_dump[i:i+32])
```

- No structure, flag format, or encoding signatures were found (e.g., no THM{ ,Base64, or printable strings)

Next I guess...

- Check if any pre-processing (e.g., compression, base64, XOR) is applied post-decryption.

Casey found the flag first:

Given file: shutdown.log-1750934543756.enc

```
└─(kali㉿kali)-[~]
```

```
└─$ file VM-Share/Crypto/shutdown.log-1750934543756.enc
```

```
VM-Share/Crypto/shutdown.log-1750934543756.enc: OpenPGP Public Key
```

```
└─(kali㉿kali)-[~]
```

```
└─$ cp VM-Share/Crypto/shutdown.log-1750934543756.enc VM-Share/Crypto/shutdown.log-1750934543756.enc.bak
```

```
└─(kali㉿kali)-[~]
```

```
└─$ ls VM-Share/Crypto
```

```
shutdown.log-1750934543756.enc  shutdown.log-1750934543756.enc.bak
```

```
└─(kali㉿kali)-[~]
```

```
└─$ gpg --output shutdown.log --decrypt VM-Share/Crypto/shutdown.log-1750934543756.enc
```

```
gpg: packet(6) with unknown version 69
```

```
└─(kali㉿kali)-[~]
```

```
└─$ gpg --list-packets VM-Share/Crypto/shutdown.log-1750934543756.enc
```

```
gpg: packet(6) with unknown version 69
```

```
# off=0 ctb=9a tag=6 hlen=5 plen=2032007829
```

```
:key packet: [unknown version]
```

```
└─(kali㉿kali)-[~]
```

```
└─$ hexdump -C VM-Share/Crypto/shutdown.log-1750934543756.enc | head
```

```
00000000  9a 79 1d fa 95 45 f6 52  0f a3 ba 55 44 40 90 69  |.y...E.R...UD@.i|
```

```
00000010  4f ed 04 a3 f2 a3 89 55  b9 46 c9 15 a3 11 08 20  |O.....U.F..... |
```

```
00000020  40 6b b9 68 98 c9 ff 0f  9f 1a c9 7b 07 d6 70 06  |@k.h.....{..p.|
```

```
00000030
```

```
└─(kali㉿kali)-[~]
```

```
└─$ openssl enc -aes-256-cbc -d -in VM-Share/Crypto/shutdown.log-  
1750934543756.enc -out shutdown.log -K $(echo -n "VIRELIA-WATER-FAC" |  
sha256sum | cut -d' ' -f1) -iv 0000000000000000000000000000000000000000000000000000000000000000
```

Decrypted log file yields flag: **THM{cbc_cl3ar4nce_gr4nt3d_10939}**